

Developer Prompts

In-place refactor: from local-dev codebase to multi-tenant production foundation. Without breaking what works.

Use with: Claude Code (recommended) or Cursor

Companion to: Immigration CRM Master Plan v1.0

Total prompts: 22 sequential prompts + 1 critical cleanup step (Prompt 15.5) across 5 phases

Run the prompts in order. Do not skip ahead. Each one assumes the previous one has been completed and committed.

How to Use This Document

This document contains **22 developer prompts** in sequence. Each prompt is a self-contained unit of work designed to be copy-pasted directly into Claude Code or Cursor. Together, they walk your existing codebase from its current single-tenant Mongo-based state to a multi-tenant Postgres-backed foundation, without losing any of the AI work you've already built.

The five phases

Phase 1 — Safety & Inventory (Prompts 01–03)

Lock down what you have. Set up branches, snapshots, and an inventory document. Zero functional changes.

Phase 2 — Foundation (Prompts 04–08)

TypeScript migration, Postgres alongside Mongo, RLS scaffolding, auth refactor. Old code keeps running; new foundation is built underneath.

Phase 3 — Module Migration (Prompts 09–15)

Migrate each business module from Mongo to Postgres, one at a time. Students, documents, AI verification, MCP, tasks, billing scaffold.

Phase 4 — Frontend Cleanup (Prompts 16–19)

Remove the MUI + PrimeReact mess. Standardize on shadcn/ui + Tailwind. Consultant and student portals get unified design language.

Phase 5 — Production-Readiness (Prompts 20–22)

Audit log, observability, deployment to AWS ca-central-1. After Prompt 22, you're ready to invite design partners.

How each prompt is structured

Every prompt has the same six-section structure so the AI knows exactly what to do, what not to touch, and how to verify success:

- **Context** — what state the codebase is in when this prompt runs.
- **Goal** — the specific outcome this prompt achieves.
- **Do not touch** — files and behaviors that must stay intact.
- **Tasks** — the ordered list of changes the AI should make.
- **Verification** — how you (the human) confirm it worked.
- **Commit message** — what to write when you save the work.

Operating rules

★ Commit after every prompt

Run the prompt. Verify it works. **Commit the changes with the suggested message.** If the next prompt breaks something, you can *git revert* back one step. Without commits, you have no safety net.

[!] Do not run multiple prompts in one session

AI IDEs lose context across long conversations. Each prompt should be a **fresh chat session**. Paste the prompt, let it work, verify, commit, close the chat, open a new one for the next prompt.

[i] When the AI gets stuck

If Claude Code or Cursor produces something wrong, *do not* argue with it across many turns. **Stop. Revert. Refine the prompt.** Add the specific constraint that was missed and start the prompt fresh. This is faster and more reliable than long debugging conversations.

Before you start

Before Prompt 01, make sure you have:

- Claude Code installed and authenticated, or Cursor with Claude Sonnet 4.6 selected.
- Your existing codebase opened in the IDE.
- Git initialized in the project root (run *git init* if not).
- Node.js 22 LTS installed.
- PostgreSQL 16 installed locally (or Docker available to run it).
- An AWS account created (you won't deploy until Prompt 22, but useful early).
- About 60–90 minutes free per prompt — most are 30 min, a few are 2+ hours.

Prompt Index

All 22 prompts plus the 15.5 cleanup step at a glance, with estimated time and dependency.

#	Title	Phase	Time
01	Codebase inventory & legacy archive	Safety	30m
02	Repository structure consolidation	Safety	45m
03	Add TypeScript to backend (incremental)	Safety	1h
04	Provision Postgres alongside Mongo	Foundation	1h
05	Auth refactor: tenant-aware JWT	Foundation	2h
06	RLS-ready schema: firms & users tables	Foundation	1.5h
07	Tenant-scoping middleware	Foundation	1h
08	Cross-tenant test harness	Foundation	1.5h
09	Migrate Student module to Postgres	Migration	3h
10	Migrate Documents + S3 module	Migration	2h
11	Migrate AI verification module (preserve prompts)	Migration	2.5h
12	Migrate Tasks module	Migration	1.5h
13	Migrate Notifications + Socket.IO	Migration	1.5h
14	Refactor MCP server for new APIs	Migration	2h
15	Add Stripe + trust ledger scaffold	Migration	3h
15.5	REMOVE MONGODB COMPLETELY (CRITICAL — do not skip)	Cleanup	30m
16	Frontend audit & UI library decision	Frontend	1h
17	Remove MUI + PrimeReact, install shadcn/ui	Frontend	2h
18	Unify consultant portal pages on shadcn	Frontend	4h
19	Unify student portal + role-aware routing	Frontend	3h
20	Tamper-evident audit log implementation	Production	2h
21	Observability: logging, Sentry, health checks	Production	1.5h
22	Deploy to AWS ca-central-1 (Terraform)	Production	4h

Phase 1 — Safety & Inventory

Three prompts. Zero functional changes. Goal: lock down what works today before changing anything. If you skip this phase you have no rollback path.

PROMPT 01 Codebase inventory & legacy archive

Create a permanent legacy branch and document everything that currently works, so we can verify nothing is lost during the refactor.

What it does

Reads through your entire backend, frontend, and MCP server, then produces a `SYSTEM_INVENTORY.md` file listing every working feature, every API endpoint, every model, and every AI prompt. This becomes your acceptance test for the entire refactor — when you reach Prompt 22, every checkmark in this file must still pass.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

I have an Immigration CRM project with 4 sub-projects:

- backend/ (Node.js + Express + MongoDB + Mongoose + OpenAI + S3 + Socket.IO)
- frontend/ (React 19 + Vite + Tailwind + mixed UI libraries: shadcn, MUI, PrimeReact)
- mcp-server/ (Node.js MCP agent exposing tools to LLMs)
- website/ (Next.js marketing site)

The codebase is local-dev only. No production data. No paying customers.

I am about to start a major refactor (Mongo to Postgres, single-tenant to multi-tenant). Before I touch anything, I need a complete inventory of what currently works so I can verify nothing is lost during the refactor.

GOAL

Create two artifacts:

1. A new git branch `legacy-archive` containing the current code, tagged v0-legacy.
2. A `SYSTEM_INVENTORY.md` file in the repo root documenting everything that works.

DO NOT TOUCH

- Do not modify any source code.
- Do not refactor anything.
- Do not delete any files.
- This prompt is read-only on the codebase except for creating new git branches and one new markdown file.

TASKS

1. Run `git status` to confirm working tree is clean. If not, stop and tell me.
2. Run `git tag v0-legacy` and `git checkout -b legacy-archive` then push the branch. Switch back to main (or master) when done.
3. Read every file in backend/ and document:
 - Every Mongoose model (Student, Task, Admin, Recommendation, Notification, File) with its fields and relationships.
 - Every Express route file with HTTP method, path, and one-line purpose.
 - Every service file with its public functions.
 - Every AI prompt string (these are critical IP - quote them verbatim).
 - All environment variables used.

4. Read every file in frontend/ and document:
 - Every React Router route with its component and access requirements.
 - Every page component (one-line purpose).
 - Which UI library each component uses (shadcn, MUI, PrimeReact, custom).
5. Read every file in mcp-server/ and document:
 - Every tool exposed by the MCP server with its name, args, and purpose.
 - The transport mechanism.
6. Output everything to a single SYSTEM_INVENTORY.md with these sections:
 - "Backend: Models"
 - "Backend: API Endpoints"
 - "Backend: Services"
 - "Backend: AI Prompts (verbatim)"
 - "Backend: Environment Variables"
 - "Frontend: Routes"
 - "Frontend: Pages"
 - "Frontend: UI Library Usage Map"
 - "MCP Server: Tools"
 - "Working End-to-End Flows" (your best inference of what works today)
 - "Known Broken or Half-Built Things"

VERIFICATION

After you finish:

- Show me the file tree of what was scanned.
- Show me a count: "X models, Y endpoints, Z services, N AI prompts."
- Confirm legacy-archive branch exists with: ``git branch | grep legacy``.
- Show me the first 50 lines of SYSTEM_INVENTORY.md.

COMMIT MESSAGE

chore: snapshot codebase state and create system inventory before refactor

- Tag v0-legacy on current main
- Create legacy-archive branch (preserved permanently)
- Add SYSTEM_INVENTORY.md documenting all working features
- Acceptance test for upcoming Mongo->Postgres + multi-tenant refactor

PROMPT 02 Repository structure consolidation

Reorganize the four sub-projects into a clean monorepo structure that supports the refactor without losing history.

What it does

Right now you have four loose folders. We turn this into a proper pnpm monorepo with shared TypeScript types, shared lint config, and a single place for the Terraform infra (added later in Prompt 22). This makes every subsequent prompt easier.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

The project has 4 sub-folders (backend, frontend, mcp-server, website) that are not unified. Each has its own package.json. There is no shared tsconfig, no shared types, no monorepo tooling. As we add TypeScript, multi-tenancy, and shared schemas across backend and MCP server, this fragmentation will

hurt us.

GOAL

Convert the project into a pnpm workspace monorepo with this structure:

```

/
  package.json           (root, type: "module", pnpm workspace)
  pnpm-workspace.yaml
  tsconfig.base.json     (shared TS settings)
  .eslintrc.cjs          (shared lint)
  .prettierrc
  apps/
    backend/             (was: backend/)
    consultant-web/      (was: frontend/)
    mcp-server/          (was: mcp-server/)
    marketing/           (was: website/)
  packages/
    shared-types/        (will hold zod schemas in Prompt 06)
    eslint-config-custom/ (placeholder for now)
  infra/                 (empty for now; Terraform lands in Prompt 22)
  docs/
    adr/                 (empty for now)
  SYSTEM_INVENTORY.md

```

DO NOT TOUCH

- Do not change any source code logic.
- Do not change any package.json dependencies (just keep them where they are).
- Do not break the ability to run each app independently.
- Use `git mv` (not plain mv) so history is preserved.

TASKS

1. Verify clean working tree.
2. Create the new directory structure.
3. `git mv` each existing app folder to its new location under apps/.
4. Create a root package.json with pnpm workspaces declared.
5. Create pnpm-workspace.yaml listing apps/* and packages/*.
6. Create tsconfig.base.json with strict: true, target: ES2022, module: NodeNext, moduleResolution: NodeNext.
7. Create root .eslintrc.cjs and .prettierrc with conservative defaults.
8. Update each app's package.json name to be scoped (e.g., @icrm/backend, @icrm/consultant-web). Do not change versions or dependencies.
9. Add a root scripts object: "dev:backend", "dev:web", "dev:mcp", "build", "lint".
10. Run `pnpm install` from the root and confirm no errors.
11. Run each app's existing dev script via `pnpm --filter @icrm/backend dev` (and same for web/mcp) and confirm each starts without error.

VERIFICATION

- `tree -L 2 -I node\_modules` shows the new structure.
- `pnpm --filter @icrm/backend dev` starts the backend.
- `pnpm --filter @icrm/consultant-web dev` starts the frontend.
- `pnpm --filter @icrm/mcp-server dev` starts the MCP server.
- All three start without errors.

```
# COMMIT MESSAGE
refactor: convert project into pnpm monorepo with apps/ and packages/

- Move backend, frontend, mcp-server, website into apps/ with git mv
- Add pnpm workspace configuration
- Add shared tsconfig.base.json, eslint, prettier
- All existing apps continue to run unchanged
```

PROMPT 03 Add TypeScript to backend (incremental)

Introduce TypeScript to the backend without rewriting existing JavaScript files yet — `allowJs` mode lets new code be TS while old code keeps working.

What it does

TypeScript is non-negotiable for the multi-tenant refactor — we need compile-time guarantees that we're filtering by `firm_id` everywhere. But we don't rewrite existing files; we configure TypeScript in `allowJs` mode so `.js` and `.ts` coexist. New work happens in `.ts`; old `.js` stays working. Files get converted as we touch them in later prompts.

PASTE THIS INTO CLAUDE CODE / CURSOR

```
# CONTEXT
The backend in apps/backend is plain JavaScript with Express 5. Going forward
we will write all new code in TypeScript, and gradually migrate existing files
as we touch them. We do NOT rewrite anything in this prompt.

# GOAL
Set up TypeScript tooling in apps/backend so:
- Existing .js files keep running unchanged.
- New .ts files compile and run.
- Both can coexist and import each other.
- `pnpm dev` starts the server with tsx watch mode.
- `pnpm build` produces a dist/ folder ready for production.

# DO NOT TOUCH
- Do not convert any existing .js file to .ts in this prompt.
- Do not change any business logic.
- Do not modify route paths or behaviors.
- Do not change Express version or any other dependency unless required by tsx.

# TASKS
1. In apps/backend, install dev dependencies:
   typescript, tsx, @types/node, @types/express, @types/jsonwebtoken,
   @types/bcrypt, @types/cors, @types/multer.
2. Create apps/backend/tsconfig.json that extends ../../tsconfig.base.json with:
   - "allowJs": true
   - "checkJs": false (we don't want to lint old JS)
   - "outDir": "./dist"
   - "rootDir": "./src"
   - "include": ["src/**/*.ts"]
   - "esModuleInterop": true
   - "resolveJsonModule": true
```


3. If the backend currently uses CommonJS (require), keep it that way - set "module": "CommonJS" in tsconfig and "type": "commonjs" in package.json. Do NOT migrate to ESM in this prompt - that's a separate change.
4. If there is no src/ directory and code lives at the root of apps/backend, move the existing JS files into src/ using `git mv`. Update any path references in package.json scripts.
5. Update apps/backend/package.json scripts:
 - "dev": "tsx watch src/index.js" (or whatever the entry file is)
 - "build": "tsc"
 - "start": "node dist/index.js"
 - "type-check": "tsc --noEmit"
6. Create one example .ts file at src/lib/typescript-canary.ts that exports a simple function. Import it from the existing index.js to prove .ts works.
7. Run `pnpm --filter @icrm/backend dev` and confirm the server starts.
8. Run `pnpm --filter @icrm/backend build` and confirm dist/ is produced.

VERIFICATION

- `pnpm --filter @icrm/backend dev` starts without errors.
- `pnpm --filter @icrm/backend build` produces dist/index.js (or equivalent).
- The existing API endpoints all still respond (test 2-3 with curl).
- The .ts canary file is imported and used in the .js entry point.

COMMIT MESSAGE

chore(backend): add TypeScript with allowJs for incremental migration

- tsx for dev watch mode
- tsc for production build
- .js and .ts files coexist
- No existing files migrated yet (later prompts handle each module)

Phase 2 — Foundation

Five prompts. This phase introduces Postgres alongside Mongo, multi-tenancy primitives, and the test harness that will catch cross-tenant data leaks. After Phase 2, the foundation is solid even though no business module has moved yet.

PROMPT 04 Provision Postgres alongside Mongo

Add Postgres + Drizzle ORM as a second database. Both run side-by-side; nothing migrates yet. This is the rails for everything that follows.

What it does

Postgres becomes available to the backend as a parallel database. Mongo stays untouched. We pick Drizzle ORM (lightweight, TypeScript-native, supports RLS cleanly). We also set up a Docker Compose for local dev so any new engineer joining the team gets Postgres + Redis with one command.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

The backend currently uses MongoDB (Mongoose). We are adding PostgreSQL 16 as a parallel database. In later prompts we migrate modules from Mongo to Postgres one at a time. Mongo keeps running for everything not yet migrated.

GOAL

- Postgres 16 available locally via Docker Compose.
- Drizzle ORM set up in apps/backend.
- A health-check endpoint that confirms both Mongo and Postgres are reachable.
- Database connection pooling configured.
- Migrations folder ready (no migrations written yet).

DO NOT TOUCH

- Do not modify any existing Mongoose models.
- Do not change any existing routes.
- Do not introduce any new business logic.

TASKS

1. At the repo root, create docker-compose.dev.yml with services:
 - postgres:16-alpine on port 5432, with volume for persistence
 - redis:7-alpine on port 6379
 - Use sensible default credentials for local dev only:
POSTGRES_USER=icrm, POSTGRES_PASSWORD=icrm_dev, POSTGRES_DB=icrm_dev
 - Add healthchecks for both.
2. Add a root script "docker:dev" that runs ``docker compose -f docker-compose.dev.yml up -d`` and "docker:down" that stops it.
3. In apps/backend, install: drizzle-orm, drizzle-kit, postgres (the pg client recommended by Drizzle), dotenv. Install drizzle-kit as devDependency.
4. Create apps/backend/src/db/postgres.ts that:
 - Reads DATABASE_URL from env (with fallback to local Docker URL).
 - Creates a pool with sensible defaults (max 20 connections, idle 30s).
 - Exports a ``db`` instance (Drizzle client) and a raw ``sql`` client.
 - Handles graceful shutdown on SIGTERM.
5. Create apps/backend/drizzle.config.ts pointing schema to `./src/db/schema/*.ts`

```

    and migrations to ./drizzle/.
6. Create apps/backend/src/db/schema/index.ts as an empty barrel file (we add
   tables in Prompt 06).
7. Create apps/backend/src/db/migrate.ts as a runnable script that applies
   pending Drizzle migrations.
8. Update the existing health-check endpoint (or create one at GET /health) to:
   - Ping Mongo (return ok/error).
   - Ping Postgres with `SELECT 1`.
   - Return JSON: { mongo: "ok|error", postgres: "ok|error", uptime: <seconds> }.
9. Update .env.example with DATABASE_URL and any new env vars.
10. Update apps/backend/package.json scripts:
    - "db:generate": "drizzle-kit generate"
    - "db:migrate": "tsx src/db/migrate.ts"
    - "db:studio": "drizzle-kit studio"

# VERIFICATION
- `pnpm docker:dev` starts Postgres and Redis containers without errors.
- `psql postgresql://icrm:icrm_dev@localhost:5432/icrm_dev -c "SELECT 1;"` works.
- `pnpm --filter @icrm/backend dev` starts the server.
- `curl http://localhost:<port>/health` returns both Mongo and Postgres "ok".
- `pnpm --filter @icrm/backend db:studio` opens Drizzle Studio in the browser.

# COMMIT MESSAGE
feat(backend): add Postgres + Drizzle alongside MongoDB

- Docker Compose for local Postgres 16 and Redis 7
- Drizzle ORM configured with empty schema
- Health endpoint reports status of both databases
- No business code touched; existing Mongo flow unchanged

```

PROMPT 05 Auth refactor: tenant-aware JWT

Modify auth so every JWT carries `firm_id` and `role`. The auth flow itself stays compatible with the current login UX.

What it does

Every request must know which firm the user belongs to. We add `firm_id` and `role` claims to the JWT. The login endpoint still works the same way for the user — they just get a richer token. We also add a temporary `default_firm` concept so existing single-tenant flows don't break before we have actual firms.

PASTE THIS INTO CLAUDE CODE / CURSOR

```

# CONTEXT
The backend has JWT-based auth. Tokens currently encode user_id and role.
For multi-tenancy we need every token to also carry firm_id. Right now
there are no firms - all data is implicitly single-tenant. We will introduce
a "default firm" so existing users transparently belong to one firm, then
build real multi-firm support in Prompt 06.

# GOAL
- Every JWT issued from this point forward includes firm_id, user_id, role.
- A default firm record exists in Postgres (we create the firms table here).

```

```
- Existing users are automatically associated with the default firm.
- Token verification middleware extracts firm_id and attaches to req.context.
- The frontend login flow still works exactly as it did before.
- Refresh tokens introduced if not already (15-minute access, 7-day refresh).

# DO NOT TOUCH
- Do not change the login UI or its API contract.
- Do not break existing /api/auth/login or /api/auth/register endpoints.
- Do not change password hashing yet (we upgrade bcrypt to argon2id later).

# TASKS
1. In apps/backend/src/db/schema/, create firms.ts with a Drizzle table:
  - id: uuid primary key, default random
  - name: text not null
  - slug: text unique not null
  - plan_tier: text default 'starter'
  - settings: jsonb default '{}'
  - created_at: timestampz default now()
  - updated_at: timestampz default now()
2. Generate the migration: `pnpm db:generate`. Review the generated SQL.
3. Apply the migration: `pnpm db:migrate`.
4. Create a one-off script at apps/backend/src/scripts/seed-default-firm.ts that:
  - Inserts a firm with name='Default Firm', slug='default'.
  - Prints the firm UUID.
  - Exits idempotently (does nothing if it already exists).
5. Run the seed script. Note the firm UUID and store it in .env as DEFAULT_FIRM_ID.
6. In the existing auth module:
  a. Modify the JWT signing function to accept and embed firm_id and role.
     Default firm_id to process.env.DEFAULT_FIRM_ID for existing users.
  b. Modify the JWT verification middleware to extract firm_id, user_id, role
     from the token and attach them to req.context = { firmId, userId, role }.
  c. If req.context already exists, throw - we want explicit shape.
7. Add refresh token support if not already present:
  - Access token: 15 min expiry, signed with JWT_ACCESS_SECRET.
  - Refresh token: 7 day expiry, signed with JWT_REFRESH_SECRET, stored as
    httpOnly cookie.
  - POST /api/auth/refresh endpoint that issues a new access token from a
    valid refresh token.
8. Add an env var JWT_REFRESH_SECRET to .env.example. Do not commit real secrets.

# VERIFICATION
- `pnpm db:studio` shows the firms table with one Default Firm row.
- Existing login still works: log in, get a token.
- Decode the token (jwt.io) and confirm it has firm_id, user_id, role.
- Hit any authenticated endpoint and confirm req.context is populated by
  adding a temporary console.log in one route handler.
- POST /api/auth/refresh returns a new access token when given a valid
  refresh token cookie.

# COMMIT MESSAGE
feat(auth): tenant-aware JWT with firm_id, refresh tokens

- Add firms table (Postgres) with Default Firm seed
```

- JWT now carries { user_id, firm_id, role }
- Auth middleware attaches req.context = { firmId, userId, role }
- Refresh token endpoint added (15m access / 7d refresh)
- Login UX unchanged for the frontend

PROMPT 06 RLS-ready schema: firms & users tables

Build the foundational Postgres tables (firms, users) with Row-Level Security policies that will enforce tenant isolation.

What it does

This prompt installs the security model that makes multi-tenancy bulletproof. Every tenant-scoped table will get a Row-Level Security policy that filters by `firm_id = current_setting('app.current_firm_id')`. If a developer ever forgets a `firm_id` filter in code, Postgres still returns nothing. This is the architectural moat.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

We have a firms table (from Prompt 05) and JWTs carrying `firm_id`. Now we add the users table in Postgres and configure Row-Level Security (RLS) so that every query is automatically filtered by `firm_id` at the database engine level. This makes cross-tenant data leakage physically impossible at the DB layer.

GOAL

- A users table in Postgres with `firm_id` foreign key.
- RLS enabled on users table with a `tenant_isolation` policy.
- A SQL helper function `set_app_context(firm_id uuid)` that the app calls at the start of each request.
- A `db.withFirmContext(firmId, callback)` helper in TypeScript that wraps any query block in a transaction with the `firm_id` set.
- A Drizzle migration script that codifies all of this.

DO NOT TOUCH

- Do not migrate the existing Mongoose User model to Postgres yet (we do that in Prompt 09 along with students).
- Do not change any existing API endpoints.
- Do not enable RLS on tables other than users in this prompt.

TASKS

1. Create `apps/backend/src/db/schema/users.ts` as a Drizzle table:
 - `id`: `uuid` primary key
 - `firm_id`: `uuid` not null, references `firms(id)` on delete cascade
 - `email`: `text` not null
 - `password_hash`: `text` not null
 - `role`: `text` not null check role in ('admin', 'senior', 'junior', 'student')
 - `first_name`: `text`, `last_name`: `text`
 - `rcic_number`: `text` (nullable, for licensed consultants)
 - `is_active`: `boolean` default true
 - `last_login_at`: `timestampz`
 - `created_at`, `updated_at`: `timestampz` default `now()`
 - unique constraint on (`firm_id`, `email`) – same email allowed across firms
2. Generate and apply the migration.

3. Create a hand-written SQL migration file at `apps/backend/drizzle/00X_rls_users.sql` with:
 - `ALTER TABLE users ENABLE ROW LEVEL SECURITY;`
 - `CREATE POLICY tenant_isolation ON users`
 `USING (firm_id = current_setting('app.current_firm_id', true)::uuid);`
 - `CREATE OR REPLACE FUNCTION set_app_context(p_firm_id uuid) RETURNS void`
 `AS $$ BEGIN PERFORM set_config('app.current_firm_id', p_firm_id::text, true);`
 `END; $$ LANGUAGE plpgsql;`
 - `GRANT EXECUTE ON FUNCTION set_app_context TO PUBLIC;`
4. Wire this hand-written SQL into the migration runner. (Drizzle's ``migrate`` will run it as a custom step. If needed, use a tagged comment like `--> statement-breakpoint` and the appropriate Drizzle pattern.)
5. In `apps/backend/src/db/postgres.ts`, add a helper:

```
export async function withFirmContext<T>(  
  firmId: string,  
  fn: (tx: typeof db) => Promise<T>  
) : Promise<T> {  
  return db.transaction(async (tx) => {  
    await tx.execute(sql`SELECT set_app_context(${firmId}::uuid)`);  
    return fn(tx);  
  });  
}
```

6. Add a unit test at `apps/backend/src/db/__tests__/rls.test.ts` that:
 - Creates two firms (A and B) and one user in each.
 - Calls `withFirmContext(A.id, () => SELECT all from users)` and asserts only A's user is returned.
 - Calls without context and asserts zero rows are returned.
 - Use `vitest`. Add `vitest` config if not already present.

VERIFICATION

- ``pnpm test`` passes the `rls.test.ts` test.
- In ``pnpm db:studio``, you can see `firms` and `users` tables and a successful `SELECT` shows zero rows from `users` (because no `app.current_firm_id` is set in the studio session - this is the expected RLS behavior).
- Connecting via `psql` with `SET app.current_firm_id` manually returns rows.

COMMIT MESSAGE

`feat(db): add users table with Row-Level Security policy`

- `users` table with `firm_id` foreign key and role enum
- RLS policy: `tenant_isolation` filters by `current_setting('app.current_firm_id')`
- `set_app_context()` SQL function for setting per-tx tenant context
- `withFirmContext()` TypeScript helper wraps queries in tenant tx
- Test proves cross-tenant queries return zero rows

PROMPT 07 Tenant-scoping middleware

Add Express middleware that automatically wraps every authenticated request in `withFirmContext`, eliminating the chance of forgetting it.

What it does

Defense in depth. Even though RLS will catch missing filters, we also make it impossible to forget at the application layer. A middleware reads `req.context.firmId` from the JWT and wraps the request in `withFirmContext()`. Any route handler that runs queries through the request-scoped `db` instance is automatically tenant-scoped.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

We have RLS enabled on the users table, `withFirmContext()` helper, and JWTs carrying `firm_id`. Now we wire a middleware so every authenticated route is automatically wrapped in tenant context without route handlers needing to remember.

GOAL

- Middleware that extracts `firmId` from `req.context` (set by auth middleware) and provides a request-scoped `db` instance with tenant context already set.
- A route handler that does ``req.db.select().from(users)`` automatically gets RLS-filtered results.
- An integration test that proves an authenticated request from Firm A only sees Firm A's data.

DO NOT TOUCH

- Do not change the auth middleware itself.
- Do not modify existing route handlers (they keep using whatever they use today).
- Do not break any current functionality.

TASKS

1. Create `apps/backend/src/middleware/tenantContext.ts`:
 - Express middleware named `tenantContextMiddleware`.
 - Reads `req.context.firmId`. If missing, returns 401 immediately (every authenticated request must have it).
 - Begins a Drizzle transaction with `set_app_context()` called inside it.
 - Attaches the transaction-scoped `db` client to `req.db`.
 - On response finish, commits the transaction. On error, rolls back.
 - Use ``res.on('finish')`` and ``res.on('close')`` to handle both happy and error paths.
 - Be careful: long-running responses (SSE, websockets) should NOT be wrapped this way. Add a comment about this and add an "exempt paths" array.
2. Create `apps/backend/src/middleware/index.ts` barrel exporting all middleware.
3. Mount the middleware AFTER the auth middleware on all `/api/*` routes that require auth. Routes like `/health`, `/api/auth/login`, `/api/auth/register` should NOT have it.
4. Add an integration test at `apps/backend/src/middleware/__tests__/tenantContext.test.ts`:
 - Create two firms with one user each.
 - Issue JWTs for both users.
 - Hit a test route that does `req.db.select().from(users)` and returns the rows.
 - Assert User A's request returns only Firm A's user.
 - Use `supertest + vitest`.

VERIFICATION

- ``pnpm test`` passes the new integration test.
- Hit `GET /api/users` (or whatever existing endpoint) with a real JWT and

```
observe the query plan in the logs - it should run inside a transaction
with set_app_context already invoked.
```

```
# COMMIT MESSAGE
```

```
feat(backend): tenant-scoping middleware wraps every authed request
```

- tenantContextMiddleware begins a tx with set_app_context(firm_id)
- Attaches tenant-scoped db to req.db for handlers to use
- Auth middleware runs first; tenant middleware runs after on /api/*
- Integration test proves Firm A sees only Firm A's data

PROMPT 08 Cross-tenant test harness

Build the automated test suite that runs in CI and fails the build if any endpoint leaks data across tenants.

What it does

This is the test that protects you forever. It enumerates every list/get endpoint on the API, sets up two firms with overlapping data, and hits each endpoint as User A asserting nothing from Firm B leaks through. New endpoints are auto-detected. If anyone ships a query that forgets *firm_id*, this test fails and the PR cannot merge.

PASTE THIS INTO CLAUDE CODE / CURSOR

```
# CONTEXT
```

```
We have RLS at the DB level and tenant context middleware at the app level.
We need an automated test that proves every API endpoint respects tenant
isolation, runs in CI on every PR, and fails the build if any leak occurs.
```

```
# GOAL
```

- A test suite at apps/backend/src/__tests__/cross-tenant.test.ts that:
 - Provisions two firms (A and B) and creates parallel data in both (users, and any other models reachable as we add them).
 - Calls every authenticated GET endpoint as User A.
 - Asserts the response contains only Firm A's data, never Firm B's.
- A GitHub Actions workflow that runs this test on every PR.
- A test fixtures helper that makes adding new endpoints to the test trivial.

```
# DO NOT TOUCH
```

- Do not change route handlers.
- Do not change RLS policies.

```
# TASKS
```

1. Create apps/backend/src/__tests__/fixtures/multitenant.ts:
 - Function provisionTwoFirmsWithData() that creates Firm A and Firm B and returns { firmA, firmB, userA, userB, jwtA, jwtB }.
 - Helper to seed N records of any type in each firm.
2. Create apps/backend/src/__tests__/cross-tenant.test.ts:
 - beforeAll: provisionTwoFirmsWithData().
 - For each known authenticated GET endpoint, write a test: "GET /api/X as User A returns only Firm A's data"
 - Endpoints to cover for now (more added as we migrate modules): GET /api/users


```
GET /api/students (if exists today; if not, a placeholder skip)
GET /api/tasks
GET /api/notifications
...whatever else exists per SYSTEM_INVENTORY.md.
- For each test, hit the endpoint with jwtA and assert:
  - Response status 200.
  - All returned items have firm_id matching firmA OR no item belongs to firmB
    (where applicable based on the response shape).
- afterAll: clean up firms (cascade deletes everything).
3. Create .github/workflows/ci.yml at the repo root:
  - Triggers on pull_request and push to main.
  - Spins up Postgres 16 service container.
  - Runs `pnpm install`, `pnpm db:migrate`, `pnpm test`.
  - Fails the build if any test fails.
4. Add a README section "Cross-Tenant Testing" in apps/backend/README.md
   explaining how to add a new endpoint to the test suite when one is built.

# VERIFICATION
- `pnpm test` passes locally.
- Push a PR (or simulate locally) and confirm the GitHub Actions workflow
  runs and passes.
- DELIBERATELY break the test by writing a query that forgets firm context:
  - Modify one endpoint to use the raw db instance instead of req.db.
  - Run `pnpm test` and confirm it FAILS.
  - Revert the deliberate break.

# COMMIT MESSAGE
feat(test): cross-tenant isolation test suite + CI

- Tests every authed GET endpoint with two-firm setup
- Asserts no data leaks between Firm A and Firm B
- GitHub Actions workflow runs on every PR
- Documented in backend README for future endpoint additions
```

Phase 3 — Module Migration

Seven prompts plus one critical cleanup step (15.5). Each prompt migrates one business module from Mongo to Postgres. Pattern is the same for each: Drizzle schema then port service logic then preserve AI prompts verbatim then switch route handlers then update tests. After Prompt 15, run **Prompt 15.5** to remove MongoDB completely from the codebase, dependencies, and Docker.

PROMPT 09 Migrate Student module to Postgres

The biggest module. Convert the Student schema (with flexible profile fields) to Postgres + JSONB.

What it does

The Student model is the heart of your CRM. We move it to Postgres with structured columns for the rigid fields and JSONB for the flexible ones (exam scores, AI-extracted fields, etc.). The Mongoose model is left in place but marked deprecated; route handlers switch to the new Drizzle-based service.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

The Student Mongoose model has a flexible profile (jsonb-like), embedded required documents with per-file AI verification status, embedded tasks, and a status field (pending -> registered -> active), plus a unique aiKey for the MCP agent.

We are migrating Student to Postgres. The structure becomes:

- students table (rigid fields)
- documents table (one row per file - extracted from embedded array)
- profile_data jsonb column on students for flexible fields
- ai_key column with unique constraint

GOAL

- New Postgres schema for students and documents.
- Service layer (Drizzle-based) implementing all student operations.
- Route handlers updated to use the new service.
- Old Mongoose model remains in place but marked @deprecated.
- Cross-tenant test extended to cover GET /api/students.
- Existing AI verification flow still works exactly as before.

DO NOT TOUCH

- Do not change AI prompt strings.
- Do not change S3 upload logic (that's Prompt 10).
- Do not modify any task-related code yet (Prompt 12).
- Do not delete the Mongoose model file - just add a JSDoc @deprecated note.

TASKS

1. Create apps/backend/src/db/schema/students.ts:
 - id: uuid pk
 - firm_id: uuid fk
 - status: text check in ('pending','registered','active','closed')
 - first_name, last_name, email
 - profile_data: jsonb default '{}'
 - ai_key: text unique not null (preserve the existing one if migrating)

```

- assigned_to: uuid fk users(id) nullable
- stage: text default 'lead' (lead/study_permit/pgwp/pr/citizenship)
- state_data: jsonb default '{}'
```

- created_at, updated_at

2. Create apps/backend/src/db/schema/documents.ts:
 - id: uuid pk
 - firm_id: uuid fk
 - student_id: uuid fk students(id) on delete cascade
 - document_type: text (passport, transcript, ielts, gic, etc.)
 - s3_key: text
 - s3_bucket: text
 - mime_type, size_bytes
 - version: integer default 1
 - file_hash: text (sha256)
 - ai_verification: jsonb default '{}'
 - uploaded_by: uuid fk users(id)
 - created_at
3. Generate and apply migration. Add hand-written RLS policies for both tables (same pattern as users).
4. Create apps/backend/src/modules/students/students.service.ts (TypeScript):
 - createStudent(input)
 - getStudentById(id)
 - listStudents(filters, pagination)
 - updateStudent(id, patch)
 - updateStudentStage(id, newStage)
 - deleteStudent(id) (soft - sets status='closed')
 - All functions accept the request-scoped db instance and rely on RLS.
 - Use zod schemas for input validation - place them in packages/shared-types/students.ts so the frontend can import them later.
5. Create apps/backend/src/modules/students/students.routes.ts replacing any existing student routes:
 - GET /api/students
 - GET /api/students/:id
 - POST /api/students
 - PATCH /api/students/:id
 - DELETE /api/students/:id
6. Mark the old Mongoose Student model with a JSDoc comment:


```
/** @deprecated Migrated to Postgres in Prompt 09. Do not use. */
```
7. Add tests for the service layer at students.service.test.ts.
8. Extend the cross-tenant test to cover the new endpoints.
9. If you have any local-dev test data in Mongo, write a one-shot ETL script at apps/backend/src/scripts/etl-students-mongo-to-pg.ts that copies it. This is optional - if you have no test data worth keeping, skip it.

VERIFICATION

- All tests pass including the extended cross-tenant test.
- Through the existing frontend (still on old UI), you can:
 - Create a student.
 - See them in the list.
 - Open their detail page.
 - All values display correctly.
- The student's ai_key is preserved (so MCP server can still find them).
- Mongoose Student model is no longer imported by any route handler

```
(grep to confirm).

# COMMIT MESSAGE
feat(students): migrate Student module to Postgres with JSONB profile

- New students and documents tables with RLS policies
- Drizzle-based service layer with zod validation
- Routes updated to use new service
- Old Mongoose model deprecated (kept temporarily for reference)
- Cross-tenant tests extended
- AI verification data preserved in documents.ai_verification jsonb
```

PROMPT 10 Migrate Documents + S3 module

Move the file upload pipeline to use the new documents table while keeping the existing S3 bucket and presigned URL logic intact.

What it does

Your S3 logic works — don't break it. We keep the same bucket, the same key strategy, the same presigned URL flow. We just point the metadata to the new documents table instead of the embedded Mongo array. We also add file hashing (SHA-256) so the audit log can prove document integrity later (Prompt 20).

PASTE THIS INTO CLAUDE CODE / CURSOR

```
# CONTEXT
The backend has working S3 upload logic via @aws-sdk/client-s3 with presigned URLs. Currently file metadata lives in an embedded array inside the Student Mongo document. We've already created a separate documents table in Postgres (Prompt 09). Now we wire the upload pipeline to write to the new table.

# GOAL
- Upload flow writes to documents (Postgres) instead of embedded Student array.
- File contents are SHA-256 hashed at upload time and stored in documents.file_hash.
- All existing AI verification triggers (Prompt 11 will refactor these) keep firing, but now read from the documents table.
- Presigned download URLs work the same way for the frontend.
- The S3 bucket structure does not change.

# DO NOT TOUCH
- Do not change S3 bucket name, region, or IAM permissions.
- Do not change the existing presigned URL TTL or signing strategy.
- Do not modify the AI verification prompts or logic (Prompt 11).
- Do not delete files from S3 yet (we'll add cleanup later).

# TASKS
1. Create apps/backend/src/modules/documents/documents.service.ts with:
  - generateUploadUrl(studentId, documentType, fileName, mimeType, sizeBytes)
    -> returns { uploadUrl, documentId, s3Key }
  - finalizeUpload(documentId, fileHash) - called by frontend after the S3 PUT succeeds. Validates the uploaded object exists, computes/stores hash, marks the row as uploaded.
```

```

- getDownloadUrl(documentId) - returns a short-TTL presigned GET URL.
- listStudentDocuments(studentId)
- deleteDocument(documentId)
2. Update or replace the existing upload route(s):
- POST /api/documents/upload-url (was probably called something different)
- POST /api/documents/:id/finalize
- GET /api/documents/:id/download-url
- DELETE /api/documents/:id
3. The frontend will need updates - DO NOT MAKE THEM HERE. Just write a short
   doc at apps/backend/docs/documents-api-changes.md describing the new flow
   so we can apply frontend changes in Prompts 18-19.
4. Wire SHA-256 hashing:
- The frontend uploads directly to S3 via presigned PUT (no change there).
- When the frontend calls finalize, the backend uses S3 HeadObject to verify
  the file exists, then GetObject to stream and hash it.
- For files larger than 50MB, do this hashing in a background job using
  BullMQ and Redis (set this up if not already - install bullmq).
5. Update the cross-tenant test to cover documents endpoints.
6. Add a service-level test for the upload flow with a mocked S3 client
   (use aws-sdk-client-mock).

# VERIFICATION
- Upload a document through the existing frontend - it should still work.
- Check the documents table in Drizzle Studio - the row exists with
  ai_verification empty (that's Prompt 11) and file_hash set.
- Download the document via the new endpoint - it works.
- The student detail page (still on old UI) still displays the document
  in their list.
- Cross-tenant test covers documents endpoints and passes.

# COMMIT MESSAGE
feat(documents): migrate document metadata to Postgres documents table

- documents.service handles upload/download/list/delete
- Files still in S3; bucket structure unchanged
- SHA-256 hashing on finalize (background job for large files)
- Frontend changes deferred to Prompts 18-19
- Cross-tenant test extended

```

PROMPT 11 Migrate AI verification module (preserve prompts)

Refactor the AI document verification service into the new architecture without changing a single character of the prompts.

What it does

Your AI prompts are domain knowledge. We preserve them verbatim. What changes is the wiring: the service reads from the new *documents* table, writes verification verdicts back to *documents.ai_verification* JSONB, and logs every call in a new *ai_jobs* table for cost tracking. The MCP server still consumes the same outputs.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

The AI module today does:

- Document classification (is this really a passport?)
- Field extraction (passport number, DOB, expiry, etc.)
- University recommendations
- Admin chat assistant (via MCP)

The prompts are working IP that took weeks to refine. We must preserve them verbatim. What changes is where data is read from and written to.

GOAL

- AI verification reads from documents (Postgres) and writes verdicts back to documents.ai_verification.
- Every AI call is logged to a new ai_jobs table with model, cost, latency, input/output (with PII redacted in the logged output).
- Prompt strings remain byte-for-byte identical to today.
- A multi-provider router is introduced (placeholder - Anthropic primary, OpenAI fallback).
- The MCP server's tools that depend on this still work.

DO NOT TOUCH

- Do not change any prompt string. Copy them verbatim from the old code.
- Do not change the AI model versions used (we'll review later).
- Do not change MCP tool schemas.

TASKS

1. Create apps/backend/src/db/schema/ai_jobs.ts:

- id: uuid pk
- firm_id: uuid fk
- job_type: text (doc_verify, field_extract, recommendation, chat, fraud_screen)
- related_entity_type: text (document/student/case)
- related_entity_id: uuid
- model: text
- provider: text (anthropic, openai)
- prompt_tokens: integer
- completion_tokens: integer
- cost_cents: numeric(10,4)
- latency_ms: integer
- status: text (pending, success, error)
- input_summary: jsonb (PII-redacted summary of input)
- output: jsonb
- error_message: text
- consultant_feedback: text (correct/incorrect/null - for the data flywheel)
- created_at: timestamptz
- Apply RLS as before.

2. Create apps/backend/src/modules/ai/ai-orchestrator.service.ts:

- Single entry point: runAIJob(jobType, input, context)
- Routes by jobType to the right model:
 - doc_verify, field_extract, fraud_screen -> claude-haiku-4-5
 - recommendation, chat -> claude-sonnet-4-6
- Wraps the call with cost calculation, latency timing, and ai_jobs logging.
- On Anthropic error, retries once on OpenAI (gpt-4o-mini for cheap, gpt-4o for mid-tier) - log which provider was used.

- Implements Anthropic prompt caching for system prompts.
- 3. Move the existing AI service files into `apps/backend/src/modules/ai/`. The actual prompt strings and parsing logic must remain unchanged. Just rewire them to call `ai-orchestrator.service.runAIJob()` instead of directly calling the OpenAI client.
- 4. Create `apps/backend/src/modules/ai/ai.routes.ts` (or modify existing):
 - `POST /api/ai/verify-document/:documentId`
 - `POST /api/ai/extract-fields/:documentId`
 - `POST /api/ai/recommendations/:studentId`
 - `POST /api/ai/chat` (the assistant entry point - delegates to MCP server as it does today; just make sure `firm_id` propagates correctly).
- 5. Update document upload flow (from Prompt 10): when a document is finalized, queue an AI verification job through BullMQ. The worker calls `ai-orchestrator.runAIJob` and updates `documents.ai_verification`.
- 6. Add a test that verifies:
 - The prompts used in the new code are identical to the originals (you can hash them and assert against expected hashes).
 - An AI verification call writes to `ai_jobs` with non-zero `cost_cents`.

VERIFICATION

- Upload a document through the frontend.
- Within 5-15 seconds, `documents.ai_verification` is populated.
- `ai_jobs` has a new row with `job_type='doc_verify'`, `cost_cents > 0`.
- Compare the verdict structure to what the old code produced - must be identical so the frontend continues to render correctly.
- Run a recommendation generation; `ai_jobs` logs it.
- Admin AI chat still works (MCP path).

COMMIT MESSAGE

feat(ai): refactor AI module with orchestrator + ai_jobs ledger

- All AI calls go through `ai-orchestrator.service`
- `ai_jobs` table tracks model, cost, latency, input/output per call
- Prompts preserved byte-for-byte (verified by hash assertion)
- Multi-provider failover (Anthropic primary, OpenAI fallback)
- Document verification triggered via BullMQ queue

PROMPT 12 Migrate Tasks module

Move the Task model (general + AI-verification subtypes) to Postgres while preserving the auto-creation triggers.

What it does

Tasks are the system's nervous system — they get auto-created when documents upload, when AI verification produces a needs-review verdict, and manually by consultants. We migrate the model to Postgres preserving both subtypes and the trigger logic.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

The Task model serves two purposes:

1. General admin-assigned tasks ("book biometrics", "send tax docs").

2. AI-verification tasks auto-created when a document needs review.

It tracks verification history. We need both subtypes preserved.

GOAL

- tasks table in Postgres with both subtypes covered via a discriminator column.
- Service layer for task CRUD + auto-creation hooks.
- Document upload (Prompt 10) and AI verification (Prompt 11) trigger task creation as before.
- Old Mongoose Task model marked @deprecated.

DO NOT TOUCH

- Do not change frontend task views yet (Prompts 18-19).
- Do not change task notification logic yet (Prompt 13).

TASKS

1. Create apps/backend/src/db/schema/tasks.ts:
 - id: uuid pk
 - firm_id: uuid fk
 - student_id: uuid fk students(id) nullable
 - document_id: uuid fk documents(id) nullable
 - task_type: text check in ('general', 'ai_verification', 'deadline', 'reminder')
 - title: text not null
 - description: text
 - status: text check in ('open', 'in_progress', 'done', 'dismissed') default 'open'
 - assigned_to: uuid fk users(id) nullable
 - created_by: uuid fk users(id) nullable (null for system-generated)
 - ai_generated: boolean default false
 - due_at: timestampz nullable
 - completed_at: timestampz nullable
 - verification_history: jsonb default '[]' (for ai_verification tasks)
 - metadata: jsonb default '{}'
 - created_at, updated_at
 - Apply RLS.
2. Create apps/backend/src/modules/tasks/tasks.service.ts with full CRUD plus:
 - createAIVerificationTask(documentId, verdict, aiJobId)
 - completeTask(id, completedByUserId)
 - reassign(id, newUserId)
3. Create tasks.routes.ts with REST endpoints.
4. Hook into the AI verification flow (Prompt 11): when a verification verdict is 'needs_review' or 'failed', auto-create an ai_verification task.
5. Hook into document upload: optionally create a 'reminder' task if the consultant configured one for that document type (this is a future feature - leave a TODO and the schema support, but don't implement the configuration UI).
6. Update cross-tenant test.

VERIFICATION

- Upload a document that AI flags as needs-review - a task auto-creates.
- Manually create a task via API - it appears in the list.
- Existing frontend task views still work (they query the same shape).

COMMIT MESSAGE

feat(tasks): migrate Task module to Postgres with subtypes

- tasks table with discriminator column (general|ai_verification|deadline|reminder)
- Service layer handles auto-creation from doc events
- AI verification 'needs_review' verdicts spawn ai_verification tasks
- Cross-tenant test extended
- Mongoose model deprecated

PROMPT 13 Migrate Notifications + Socket.IO

Move notifications to Postgres and ensure Socket.IO continues to push real-time events scoped to the firm.

What it does

Real-time push is core UX. We move the persistence to Postgres and make Socket.IO room-scoped by firm so a firm's events never reach another firm's connected clients. This is the Socket.IO equivalent of RLS.

PASTE THIS INTO CLAUDE CODE / CURSOR

```
# CONTEXT
Notifications are stored in Mongo and pushed in real-time via Socket.IO.
We need to migrate persistence and tighten Socket.IO so events are
firm-scoped (no cross-tenant leakage even at the websocket layer).

# GOAL
- notifications table in Postgres.
- Socket.IO uses Redis adapter (multi-instance ready).
- Each connected client joins a room named firm:<firmId>.
- All emit() calls target the firm room, never the entire connected set.
- Clients reconnecting validate their JWT before joining the firm room.

# DO NOT TOUCH
- Do not change the frontend notification UI.
- Do not change the email service yet (will do in Prompt 15 with Stripe).

# TASKS
1. Create apps/backend/src/db/schema/notifications.ts:
  - id: uuid pk
  - firm_id: uuid fk
  - user_id: uuid fk users(id) (recipient)
  - type: text (document.uploaded, task.assigned, ai.verified, billing.paid, etc.)
  - title: text
  - body: text
  - payload: jsonb
  - read_at: timestamptz nullable
  - delivered_via: text[] (in_app, email, sms, whatsapp)
  - created_at
  - Apply RLS.
2. Create apps/backend/src/modules/notifications/notifications.service.ts:
  - send(firmId, userId, type, title, body, payload)
  - Inserts into Postgres.
  - Emits via Socket.IO to firm:<firmId>:user:<userId>.
  - Optionally enqueues email job.
```

```

- markAsRead(id)
- listForUser(userId, { unreadOnly, pagination })
3. Configure Socket.IO Redis adapter:
- Install @socket.io/redis-adapter and ioredis.
- In the Socket.IO setup, attach the Redis adapter using the existing Redis
  connection (from BullMQ).
4. Authenticate Socket.IO connections:
- Client sends JWT in handshake.
- Server validates and joins the socket to firm:<firmId> and
  firm:<firmId>:user:<userId> rooms.
- Reject unauthenticated connections.
5. Replace any io.emit() calls (broadcast to all) with io.to(roomName).emit()
  calls scoped to the firm. Audit the codebase to ensure no broadcast remains.
6. Update cross-tenant test to cover notifications endpoints.

# VERIFICATION
- Connect two browser tabs as users from different firms.
- Trigger a notification in Firm A; only Firm A's tab receives it.
- Reload the tab; notifications persist (loaded from Postgres on connect).
- A notification marked read in one tab updates in real-time in another tab
  for the same user.

# COMMIT MESSAGE
feat(notifications): migrate to Postgres + firm-scoped Socket.IO rooms

- notifications table with payload jsonb
- Socket.IO uses Redis adapter for multi-instance
- Clients join firm:<firmId> rooms after JWT validation
- All io.emit() calls scoped; no more global broadcasts
- Cross-tenant test extended

```

PROMPT 14 Refactor MCP server for new APIs

Update the MCP server's tool implementations to call the new Postgres-backed services. Keep the tool schemas unchanged.

What it does

The MCP server's tool definitions (their names, inputs, outputs) are a contract with the LLM. We don't change the contract — we change the implementation. Each tool that previously hit Mongo now hits the new Postgres-backed service via the same backend HTTP API. The admin AI chat experience is unchanged.

PASTE THIS INTO CLAUDE CODE / CURSOR

```

# CONTEXT
The MCP server exposes tools to the LLM (getStudentById, searchStudents,
uploadDocument, etc.). The tool schemas are part of a contract with the LLM
and must not change. Implementations need to call the new Postgres-backed
backend services.

# GOAL
- All MCP tools work end-to-end against the new backend.

```

```

- Tool schemas (names, JSON schemas for inputs and outputs) are byte-identical.
- The MCP server gets firm context from a service-account JWT or a per-call
  firm_id parameter (admin chat is always scoped to one firm).
- Add a typescript types package shared between backend and MCP for tool
  payloads.

# DO NOT TOUCH
- Do not change tool names.
- Do not change tool input schemas.
- Do not change the MCP transport (stdio or HTTP - whatever you use today).

# TASKS
1. Migrate apps/mcp-server to TypeScript using the same allowJs approach as
  Prompt 03 if it isn't already TS.
2. Create packages/shared-types/mcp.ts with zod schemas for every tool's
  input and output. Generate JSON Schema from these via zod-to-json-schema.
3. Compare generated schemas to the existing hand-written ones. Reconcile
  any differences in favor of the existing schemas (the LLM has been trained
  on these).
4. Refactor each tool implementation:
  - Take an HTTP-based approach: each tool calls the backend's REST API
    using axios with a service JWT that includes the firm_id of the admin
    who initiated the chat.
  - The chat session passes firm_id when starting; MCP propagates it.
5. Add a test for each tool that mocks the backend and verifies inputs/outputs
  match the schema.
6. Update apps/backend/src/modules/ai/assistant.service.ts (or equivalent
  bridge) to pass firm_id to the MCP server when starting a chat session.

# VERIFICATION
- Open the admin AI assistant in the existing frontend.
- Ask "what documents is John missing?" and confirm the answer is correct
  for John's actual document state in Postgres.
- Ask the assistant to do a write operation (e.g., update student stage)
  and confirm it completes and the change is reflected.
- Try the assistant from a different firm's account - it should only see
  that firm's data, never the first firm's.

# COMMIT MESSAGE
refactor(mcp): wire tools to new Postgres backend; keep schemas unchanged

- All tools now call backend REST API with firm-scoped service JWT
- Tool schemas verified identical to legacy via hash check
- Shared zod schemas in @icrm/shared-types
- MCP server migrated to TypeScript

```

PROMPT 15 Add Stripe + trust ledger scaffold

Introduce Stripe Connect and the trust ledger schema. This is new functionality, not a migration — it sets up the moat feature for billing.

What it does

This prompt is additive — there's nothing to migrate because the feature didn't exist before. We add Stripe Connect Custom accounts (one per firm, used as their trust account), an invoices table, a trust_ledger table, and the retainer agreement scaffold. The UI for this is built later in the frontend phase.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

The CRM doesn't have payments today. We're adding Stripe + trust accounting + e-signed retainers as one of the moat features. This prompt sets up the backend scaffold; the frontend pieces come later.

GOAL

- Stripe Connect Custom integration: each firm has a Stripe account.
- Invoices, trust_ledger, retainer_agreements tables in Postgres.
- Webhook endpoint for Stripe events with signature verification.
- Service layer for creating invoices, recording trust deposits/withdrawals, and reconciliation reports.
- Retainer template engine with Handlebars-style interpolation.

DO NOT TOUCH

- Do not build the frontend UI for this yet.
- Do not change any existing routes.

TASKS

1. Install stripe SDK in apps/backend.
2. Add Stripe-related env vars to .env.example: STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_CONNECT_CLIENT_ID. Use test keys.
3. Create schemas:
 - a. firm_stripe_accounts:
 - firm_id pk fk
 - stripe_account_id text unique
 - operating_account_id text
 - trust_account_id text (separate Stripe account for client funds)
 - onboarding_status text
 - charges_enabled bool
 - created_at
 - b. invoices:
 - id, firm_id, student_id, retainer_id (nullable)
 - amount_cents integer, currency text default 'CAD'
 - status text check in ('draft','sent','paid','void','refunded')
 - stripe_invoice_id, stripe_payment_intent_id
 - issued_at, paid_at, due_at
 - line_items jsonb
 - c. trust_ledger:
 - id, firm_id, student_id (nullable)
 - amount_cents integer (positive = deposit, negative = withdrawal)
 - entry_type text check in ('deposit','withdrawal','transfer_to_operating','refund')
-)
 - related_invoice_id, related_stripe_charge_id
 - description text, recorded_by uuid fk users(id)
 - reconciled boolean default false
 - created_at
- d. retainer_agreements:

- id, firm_id, student_id, version int
- template_key text (e.g., 'study_permit_v1')
- rendered_html text
- signed_at, signed_ip, signed_user_agent
- pdf_s3_key text (immutable signed copy in S3)
- scope jsonb (services covered, fees, etc.)
- status text check in ('draft', 'sent', 'signed', 'withdrawn')
- All have firm_id and RLS.

4. Create modules:

- apps/backend/src/modules/billing/stripe.service.ts
 - createConnectAccount(firmId)
 - createOperatingAndTrustAccounts(firmId)
 - getOnboardingLink(firmId)
- apps/backend/src/modules/billing/invoices.service.ts
 - createInvoice(studentId, lineItems)
 - sendInvoice(id) - creates Stripe Invoice and emails student
- apps/backend/src/modules/billing/trust.service.ts
 - recordDeposit, recordWithdrawal, recordTransfer
 - generateMonthlyReconciliation(firmId, year, month) - returns a report comparing trust_ledger sums to Stripe trust account balance
- apps/backend/src/modules/billing/retainers.service.ts
 - listTemplates() - reads from a templates/ directory; ship 3 starter templates: study_permit_v1.hbs, pgwp_v1.hbs, generic_v1.hbs
 - createDraft(studentId, templateKey, scope)
 - sendForSignature(id) - emails student a signed link
 - finalizeSignature(id, signature, ip, userAgent) - stores signed PDF in S3 with Object Lock (so it's immutable)

5. Create routes for all of the above under /api/billing/*.

6. Create POST /api/webhooks/stripe with signature verification using the raw body. Handle invoice.paid, payment_intent.succeeded, account.updated events.

7. Add tests for invoice creation, trust ledger entry, retainer signing flow (using a Stripe mock).

8. Extend cross-tenant test for all new endpoints.

VERIFICATION

- A Stripe Connect onboarding URL is generated for the Default Firm.
- Manually complete Stripe test onboarding (in test mode).
- Create an invoice via API; pay it via Stripe test card; trust_ledger records a deposit.
- Generate a retainer draft, simulate signing; the signed PDF lands in S3.
- Run the monthly reconciliation; values match.

COMMIT MESSAGE

feat(billing): Stripe Connect + trust ledger + retainer scaffolding

- firm_stripe_accounts, invoices, trust_ledger, retainer_agreements tables
- stripe.service handles Connect Custom onboarding (operating + trust)
- retainers.service with Handlebars templates and S3 Object Lock storage
- Webhook endpoint with signature verification
- 3 starter retainer templates (study_permit, pgwp, generic)
- Frontend UI deferred to Prompts 18-19



■ CRITICAL CLEANUP STEP · DO NOT SKIP · RUN AFTER PROMPT 15**PROMPT 15.5 Remove MongoDB completely**

Tear out every trace of MongoDB from the codebase, dependencies, Docker, and environment in a single decisive commit. Postgres becomes the sole datastore.

Why this is its own prompt

During Prompts 09 through 15, each Mongoose model was marked **@deprecated** but kept in the codebase as a reference during migration. Mongo also kept running in Docker. This was deliberate — it gave you a safety net while modules were being ported one at a time.

Now that all migration is complete, Mongo is dead weight: deprecated files lying around, unused dependencies bloating your install, a Docker container nobody talks to, and environment variables nobody reads. **This prompt removes all of it in one decisive commit**, producing a clean before/after line in your git history that says "MongoDB is gone."

Run this BEFORE starting Phase 4

Do not begin frontend cleanup (Prompt 16) while Mongo is still installed. Frontend work assumes a clean Postgres-only backend. Running 15.5 first also gives you a moment to celebrate a major milestone — the database migration is officially complete.

★ Your historical Mongo code is safe forever

Remember: the entire pre-refactor codebase, including all original Mongoose models, lives on the **legacy-archive** branch from Prompt 01. This deletion only removes Mongo from the *active* codebase. If you ever need to look at the old schema, run **git checkout legacy-archive** and it's all there. You are cleaning the active project, not destroying history.

PASTE THIS INTO CLAUDE CODE / CURSOR**# CONTEXT**

We have completed migration of every module from MongoDB to Postgres (Prompts 09 through 15). The Mongoose models still exist in the codebase marked @deprecated. Mongo is still running in docker-compose.dev.yml. No application code should be importing or querying it anymore. We now verify that and remove Mongo completely in a single commit.

GOAL

- Confirm nothing in the codebase imports Mongoose or any Mongo-related package.
- Delete all deprecated Mongoose model files.
- Remove mongoose, mongodb, and any related packages from dependencies.
- Remove MongoDB connection setup code.
- Remove the mongo service from docker-compose.dev.yml.
- Update SYSTEM_INVENTORY.md to mark Mongo as fully removed.
- Health check no longer reports Mongo status.

DO NOT TOUCH

- Do not delete the legacy-archive branch (the Mongo code is preserved there forever as the historical reference).
- Do not modify any Postgres-related code.

- Do not change any business logic.

TASKS

1. Static verification first - make sure nothing depends on Mongo:
 - a. Run: `grep -rn "mongoose" apps/ packages/ --include="*.ts" --include="*.js"`
Expected: only matches in deprecated model files we are about to delete.
If any non-deprecated file still imports mongoose, STOP and tell me which one - that means a migration step was missed.
 - b. Run: `grep -rn "from 'mongodb'" apps/ packages/`
Expected: zero matches (or only in deprecated files).
 - c. Run: `grep -rn "MongoClient\|connectDB\|mongoose.connect" apps/ packages/`
Expected: only in the connection setup file we are about to delete.
2. Delete the deprecated Mongoose model files. Based on the inventory:
 - apps/backend/src/models/Student.js (or wherever the Mongoose Student lives)
 - apps/backend/src/models/Task.js
 - apps/backend/src/models/Admin.js
 - apps/backend/src/models/Recommendation.js
 - apps/backend/src/models/Notification.js
 - apps/backend/src/models/File.js
 - Any other Mongoose models in src/models/.Use `git rm` so the deletion is tracked.
3. Delete the Mongo connection setup file (typically `src/config/db.js` or similar that calls `mongoose.connect`). Use `git rm`.
4. Remove from `apps/backend/package.json`:
 - "mongoose"
 - "mongodb" (if present)
 - Any mongo-specific plugins like "mongoose-paginate" etc.Run `pnpm install` to update the lockfile.
5. Search for any environment variables related to Mongo (`MONGO_URI`, `MONGODB_URL`, etc.). Remove from `.env.example`. If they are documented in any README, remove those references too.
6. Update `apps/backend/src/index.ts` (or main entry file) - remove any Mongo connection initialization. The boot sequence should now only initialize Postgres + Redis.
7. Update the `/health` endpoint:
 - Remove the mongo status field from the response.
 - Now reports only `{ postgres, redis, uptime }`.
8. Remove the mongo service from `docker-compose.dev.yml` entirely. Remove the persistent volume too.
9. Update `SYSTEM_INVENTORY.md`:
 - Add a section "MongoDB Removal" at the top with the date and a note that all data has been migrated to Postgres.
 - Mark every Mongoose-model line item as REMOVED.
10. Run the full test suite: `pnpm test`. Every test must pass.
11. Run `pnpm --filter @icrm/backend build`. Must succeed.
12. Run `pnpm --filter @icrm/backend dev` and confirm the server starts cleanly with no Mongo-related warnings or errors.
13. Run `docker compose -f docker-compose.dev.yml down -v` to clean up the Mongo container and volume. Then `docker compose -f docker-compose.dev.yml up -d` to bring the new (Mongo-free) stack online.
14. Smoke test through the running app:
 - Log in.
 - Create a student.

- Upload a document.
 - Generate AI verification.
 - Generate a recommendation.
 - Run the AI assistant chat.
- All of these must work without any Mongo errors in the logs.

VERIFICATION

- `grep -rn "mongoose|mongodb" apps/ packages/` returns zero results (or only in lockfile entries that pnpm will clean up).
- `pnpm list --filter @icrm/backend mongoose` returns nothing.
- `docker compose ps` shows only postgres and redis containers.
- `/health` endpoint returns `{ postgres: "ok", redis: "ok", uptime: <n> }` with no mongo field at all.
- All cross-tenant tests pass.
- Smoke test of the live app works end-to-end.

COMMIT MESSAGE

chore: remove MongoDB completely; Postgres is now sole datastore

- Delete all deprecated Mongoose model files
- Remove mongoose and mongodb from dependencies
- Remove Mongo connection setup and env vars
- Remove mongo service from `docker-compose.dev.yml`
- `/health` no longer reports Mongo status
- Legacy code preserved in `legacy-archive` branch
- All migrations from Prompts 09-15 verified by smoke test

[!] If grep finds active Mongoose imports, STOP

If step 1 of the prompt finds any non-deprecated file still importing mongoose, that means a migration step was missed somewhere in Prompts 09 through 15. **Do not delete the model files yet.** Identify which module is still on Mongo, run a targeted migration prompt for that one, then come back to 15.5. The deletion only happens when grep is clean.

Phase 4 — Frontend Cleanup

Four prompts. We remove the UI library mess (MUI + PrimeReact + shadcn all coexisting) and standardize on shadcn/ui + Tailwind. We also unify the consultant and student portals on a consistent design language. This is the most visible improvement; users will feel it immediately.

PROMPT 16 Frontend audit & UI library decision

Inventory the frontend's component usage across MUI, PrimeReact, shadcn, and custom code. Produce a migration plan.

What it does

Before deleting anything, we map what's actually being used. The output is a migration spreadsheet: for every page, every MUI/PrimeReact component used, the shadcn equivalent, and the migration complexity. This stops us from accidentally deleting components that are core to an existing flow.

PASTE THIS INTO CLAUDE CODE / CURSOR

```
# CONTEXT
The frontend at apps/consultant-web uses three UI libraries simultaneously:
MUI, PrimeReact, and shadcn/ui. Plus Framer Motion for animations and
Heroicons + Lucide for icons. This is unsustainable and bloats bundle size.
We need to consolidate on shadcn/ui + Tailwind + Lucide before doing any
serious UI work.

# GOAL
- A FRONTEND_AUDIT.md file at the repo root documenting:
  - Every page and which UI library each uses for each major component.
  - Every shared component and its UI library origin.
  - A bundle size report (current).
  - A migration table mapping MUI/PrimeReact components to shadcn equivalents.
  - An ordered migration plan (which pages migrate first, last).
- No code changes in this prompt - read-only audit.

# DO NOT TOUCH
- Do not delete or modify any frontend code in this prompt.
- Do not install or uninstall packages.

# TASKS
1. Read apps/consultant-web/src/ recursively.
2. For each page component in src/pages/ (or wherever they live), list
   the MUI/PrimeReact/shadcn components used.
3. For each shared component in src/components/, do the same.
4. Run `pnpm --filter @icrm/consultant-web build` and capture the bundle
   size report. Note current size.
5. Build the migration table:
   | MUI/Primereact component | Used N times | shadcn equivalent | Notes |
6. Propose an ordered migration plan (least-coupled pages first).
7. Output everything to FRONTEND_AUDIT.md.

# VERIFICATION
- FRONTEND_AUDIT.md exists with at least:
```

```

- 1 page-level inventory table.
- 1 component migration mapping table.
- 1 ordered migration plan with rationale.
- No other files have been modified.

# COMMIT MESSAGE
docs: frontend UI library audit and consolidation plan

```

PROMPT 17 Remove MUI + PrimeReact, install shadcn/ui

Begin the UI consolidation. Install full shadcn/ui kit; remove MUI and PrimeReact. Build will break — that's expected.

What it does

The first deliberate-break prompt. After this, every page that imported MUI or PrimeReact will fail to build. That's the point — we've made the scope of the cleanup explicit. The next two prompts (18 and 19) fix every broken page.

PASTE THIS INTO CLAUDE CODE / CURSOR

```

# CONTEXT
We have FRONTEND_AUDIT.md (from Prompt 16) listing every MUI and PrimeReact
usage. Now we remove these libraries and install the full shadcn/ui set.
This will break every page using them. Prompts 18-19 fix the breakage.

# GOAL
- MUI and PrimeReact uninstalled.
- shadcn/ui fully installed with its standard component set.
- Tailwind config aligned with shadcn requirements.
- Lucide React kept; Heroicons removed (consolidate on Lucide).
- Framer Motion kept (it doesn't conflict).
- Build is BROKEN at the end of this prompt - that is expected.

# DO NOT TOUCH
- Do not start fixing broken pages yet.
- Do not modify business logic anywhere.

# TASKS
1. In apps/consultant-web:
  - Uninstall: @mui/material, @mui/icons-material, @mui/x-*, primereact,
    primeicons, @heroicons/react.
  - Confirm shadcn/ui is initialized (npx shadcn@latest init if not).
  - Install the full set of shadcn components most likely to be used:
    button, input, label, textarea, select, dialog, dropdown-menu,
    toast (sonner), table, card, badge, avatar, alert, tabs, separator,
    checkbox, radio-group, switch, calendar, popover, command, scroll-area,
    skeleton, tooltip, sheet, form (react-hook-form + zod resolver).
  - Use the official shadcn CLI to install each, e.g.:
    npx shadcn@latest add button input label ...
2. Update tailwind.config.js to reflect the shadcn theme. Ensure the
  content array includes "./src/**/*.{ts,tsx,js,jsx}".
3. Update src/index.css with the shadcn CSS variables (light + dark).
4. Configure react-hook-form + zod for forms (install if missing).

```

```

5. Set up Sonner for toasts (replaces any toast library used today).
6. Run `pnpm --filter @icrm/consultant-web build` - it WILL fail on imports
   from @mui and primereact. Capture the list of failing files in
   FRONTEND_BREAKAGE.md so Prompts 18-19 know what to fix.

# VERIFICATION
- Old packages are gone: `pnpm list --filter @icrm/consultant-web @mui/material primereact`
  returns nothing.
- New shadcn components exist under src/components/ui/.
- Build fails (expected) and FRONTEND_BREAKAGE.md lists the failing files.

# COMMIT MESSAGE
chore(web): remove MUI + PrimeReact; install full shadcn/ui set

- Uninstall @mui/*, primereact, @heroicons/react
- Install shadcn/ui standard components via CLI
- Tailwind theme + CSS variables aligned to shadcn
- Build is intentionally broken; FRONTEND_BREAKAGE.md tracks fix list
- Prompts 18-19 will fix all pages

```

PROMPT 18 Unify consultant portal pages on shadcn

Migrate every consultant-portal page from old UI libraries to shadcn. After this prompt, the consultant app builds and runs cleanly.

What it does

Page by page, we replace MUI/PrimeReact components with shadcn equivalents using the mapping from Prompt 16. We also wire the frontend to the new backend API contracts from Prompts 09–15 (students, documents, AI verification, tasks, billing). This is the longest single prompt — expect to spend half a day.

PASTE THIS INTO CLAUDE CODE / CURSOR

```

# CONTEXT
shadcn/ui is installed (Prompt 17). FRONTEND_BREAKAGE.md lists every file
that fails to build because it imports from @mui or primereact. Backend
APIs have changed in Prompts 09-15 - some response shapes are different.
We need to fix all of this.

# GOAL
- Every consultant-portal page builds and runs.
- Every page uses shadcn components consistently.
- Every page calls the new backend APIs with correct payloads.
- A consistent design language across the app: same spacing, same colors,
  same form patterns.
- TanStack Query (React Query) for data fetching - install if not already used.
- Zustand or Jotai for any client state - keep what you have if it works.

# DO NOT TOUCH
- Do not migrate the student portal yet (Prompt 19).
- Do not change route paths.

```

```
- Do not change page-level access control.

# TASKS
1. Install @tanstack/react-query if missing. Set up a QueryClientProvider
   at the app root with sensible defaults.
2. Create src/lib/api.ts as a typed API client:
   - Wraps fetch with auth (read JWT from secure storage).
   - Refresh token logic on 401.
   - Type-safe methods generated from the @icrm/shared-types zod schemas.
3. For each page in FRONTEND_BREAKAGE.md (consultant-side only), in the
   order proposed by FRONTEND_AUDIT.md:
   a. Replace MUI/PrimeReact components with shadcn equivalents.
   b. Replace any direct fetch/axios calls with TanStack Query hooks
      consuming the new API client.
   c. Update payload shapes to match the new backend contracts.
   d. Ensure forms use react-hook-form + zod resolver against the shared
      schemas.
   e. Run `pnpm dev` and visually verify the page renders correctly.
4. Pages to handle (consultant-portal only):
   - Login, Register, Forgot Password
   - Admin Dashboard
   - Student List, Registered Students, Pending Students
   - Create Student
   - Student Detail (the largest page; tabbed: profile, documents, tasks,
     comms, audit)
   - Contact Requests
   - Tasks list
   - Notifications
   - AI Assistant chat
5. Replace old toast usages with Sonner.
6. Replace old icon imports (Heroicons) with Lucide.
7. Run lint, type-check, build. All must pass.

# VERIFICATION
- `pnpm --filter @icrm/consultant-web build` succeeds.
- `pnpm --filter @icrm/consultant-web dev` runs.
- Manually walk through every consultant-portal page; everything renders
  and basic actions work (create student, upload document, view AI verdict,
  send a task, etc.).
- Bundle size is dramatically smaller than the pre-Prompt-17 number
  (typically 30-50% reduction).

# COMMIT MESSAGE
refactor(web): consultant portal migrated to shadcn/ui + new backend APIs

- All consultant pages use shadcn components
- TanStack Query for data fetching with typed API client
- react-hook-form + zod for all forms
- Sonner toasts; Lucide icons
- Bundle size reduced by ~XX% (verified)
```

PROMPT 19 Unify student portal + role-aware routing

Same treatment for the student portal. Plus: cleanup of routing so role-based access is centralized.

What it does

Student portal pages get the shadcn migration. We also refactor the router so role checks live in one place (a route guard component) instead of being duplicated across every page. This makes future RBAC additions (Prompt 20+ era) trivial.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

The consultant portal is migrated (Prompt 18). The student portal is still on the old UI mess. Also, role checks (admin vs student) are scattered across pages. We fix both here.

GOAL

- Every student-portal page uses shadcn.
- A single RouteGuard component handles all role-based access.
- Student portal pages use the new APIs and shapes.
- The /admin and /student route trees are organized so the RouteGuard enforces role at the layout level, not per-page.

DO NOT TOUCH

- Do not change consultant-portal pages (already done).
- Do not introduce SSO yet (deferred per master plan).

TASKS

1. Create src/components/auth/RouteGuard.tsx:
 - Props: { allowedRoles: Role[], children }.
 - Reads role from auth context.
 - Redirects to /login if unauthenticated.
 - Redirects to /403 if role not allowed.
2. Refactor the router so:
 - /admin/* is wrapped in `<RouteGuard allowedRoles={['admin','senior','junior']}>`.
 - /student/* is wrapped in `<RouteGuard allowedRoles={['student']}>`.
 - Public pages are not wrapped.
 - Inside admin pages, fine-grained role checks (e.g., a junior cannot delete) are still per-component but use a `useRole()` hook.
3. Migrate every student-portal page to shadcn:
 - Student Dashboard
 - Profile (with editable fields backed by zod)
 - Documents (upload + list, hooked to new presigned URL flow)
 - Tasks
 - University Recommendations (read-only AI output viewer)
 - Change Password
 - View / sign retainer (new from Prompt 15)
 - Pay invoice (new from Prompt 15; redirect to Stripe Checkout)
4. Add a 403 Forbidden page.
5. Add a NotFound page.
6. Run all checks; build must succeed.

VERIFICATION

- Every student page renders.
- An admin trying to access /student/* gets redirected to 403 (or to /admin).
- A student trying to access /admin/* gets redirected.
- Document upload from student portal works end-to-end (upload, AI verifies, result appears).
- A student can pay a test invoice through Stripe test mode.

COMMIT MESSAGE

refactor(web): student portal on shadcn; centralized RouteGuard

- All student pages migrated to shadcn
- RouteGuard at /admin and /student layout level
- useRole() hook for in-page fine-grained checks
- 403/404 pages added
- Student-side billing flow (view + sign retainer, pay invoice) wired

Phase 5 — Production-Readiness

Three prompts. Audit log, observability, and AWS deployment. After Prompt 22, the system is ready for design partners to use on real cases.

PROMPT 20 Tamper-evident audit log implementation

Build the hash-chained, append-only audit log that powers the CICC Inspection Packet feature.

What it does

Implements moat feature #3. Every state-changing action writes an audit_log row with a SHA-256 hash linking it to the previous entry. A daily verification job confirms the chain is intact. The chain head is published to S3 with Object Lock as an offline tamper-evidence anchor. We also build the one-click CICC Inspection Packet generator.

PASTE THIS INTO CLAUDE CODE / CURSOR

```
# CONTEXT
CICC compliance requires accurate, retained, tamper-evident records of
every client interaction. We implement this as a hash-chained append-only
audit log per firm.

# GOAL
- audit_log table with hash-chained entries.
- A withAudit() helper wrapping any state-changing service call to auto-log.
- A daily verification job that walks each firm's chain and alerts on breaks.
- The chain head hash for each firm is written daily to S3 with Object Lock.
- A one-click endpoint that produces a CICC Inspection Packet PDF for a
  given client file: full retainer, every document version, every audit entry.

# DO NOT TOUCH
- Do not change existing service signatures - withAudit wraps them.

# TASKS
1. Create apps/backend/src/db/schema/audit_log.ts:
  - id uuid pk
  - firm_id uuid fk
  - actor_id uuid (or 'system')
  - action text (e.g., 'document.upload', 'case.stage.transition')
  - entity_type text, entity_id uuid
  - payload jsonb (PII redacted; only metadata)
  - payload_hash text (sha256 of canonical JSON of payload)
  - prev_hash text (entry_hash of previous entry in this firm's chain)
  - entry_hash text (sha256(payload_hash || prev_hash))
  - signature text (hmac-sha256 with kms-rotated key)
  - created_at timestamptz default now()
  - REVOKE UPDATE, DELETE on this table from app role - INSERT only.
  - Apply RLS.

2. Create apps/backend/src/modules/audit/audit.service.ts:
  - logEvent(firmId, actorId, action, entityType, entityId, payload)
  - verifyFirmChain(firmId) - returns { valid: bool, brokenAt: id|null }
  - publishChainHead(firmId) - writes the latest entry_hash to
    s3://audit-anchors/<firmId>/<date>.txt with Object Lock.
```



```

- generateCICCIInspectionPacket(studentId) - produces a signed PDF
  containing:
  a. Cover page with firm + client metadata.
  b. Retainer agreement(s).
  c. Every document with version history and ai_verification verdicts.
  d. Every communication.
  e. Every audit_log entry for this client.
  f. Signature page with the chain-head hash.
- Store the generated packet in S3 with Object Lock.
3. Create a withAudit() higher-order function:
  export const withAudit =
    (action: string, entityType: string) =>
      <T extends (...args: any[]) => Promise<any>>(fn: T): T => { ... }
  - Wraps any service function; on success, logs the event automatically.
4. Apply withAudit to the most important service methods (start with: student
  create/update/delete, document upload/finalize/delete, case stage transition,
  retainer signing, invoice paid, trust ledger entries, AI verification verdicts).
5. Set up a cron job (BullMQ repeatable job) to run verifyFirmChain for every
  firm daily at 2 AM, plus publishChainHead.
6. Add an admin route GET /api/clients/:studentId/inspection-packet that
  returns the presigned URL of the generated PDF (background job; status
  endpoint).
7. Tests:
  - Logging an event produces a chain entry with valid hashes.
  - Tampering with a row breaks verification.
  - Inspection packet generation produces a non-empty PDF.

# VERIFICATION
- After multiple actions in the system, audit_log has many rows.
- verifyFirmChain returns valid=true.
- Manually update an audit_log row (DBA bypass via psql); verification returns
  valid=false with the first broken entry.
- Run /api/clients/:id/inspection-packet for a real student; download and
  open the PDF; it contains everything expected.
- Chain head appears in the audit-anchors S3 bucket.

# COMMIT MESSAGE
feat(audit): tamper-evident audit log + CICC Inspection Packet

- audit_log table with hash-chained entries (RLS, INSERT-only)
- withAudit() wraps state-changing services for auto-logging
- Daily chain verification job + S3 Object Lock chain anchor
- One-click CICC Inspection Packet PDF generator
- Tampering test confirms breakage detection

```

PROMPT 21 Observability: logging, Sentry, health checks

Wire up structured logging, error tracking, and SLO metrics so production failures are visible within 60 seconds.

What it does

You can't operate what you can't see. We wire pino for structured JSON logs, Sentry for error tracking, and OpenTelemetry traces. Health and readiness endpoints are formalized for the load balancer. Per-request latency and AI cost get exported as metrics.

PASTE THIS INTO CLAUDE CODE / CURSOR

CONTEXT

Before production, we need observability so issues are visible immediately. We use pino (structured logs), Sentry (error tracking), OpenTelemetry (traces), and basic SLO dashboards (Grafana Cloud free tier is fine for now).

GOAL

- All console.log replaced with pino structured logger.
- Every request carries a request_id propagated through logs and traces.
- Sentry captures unhandled errors in backend and frontend.
- /health and /ready endpoints distinguished correctly.
- A handful of SLO metrics exported (api latency p50/p95/p99, AI cost per request, db query latency).
- A runbook documenting how to investigate common issues.

DO NOT TOUCH

- Do not break any existing endpoint.

TASKS

1. Install pino, pino-http in apps/backend.
2. Create apps/backend/src/lib/logger.ts:
 - Pretty format in dev, JSON in prod.
 - Includes request_id, firm_id, user_id when available.
3. Create middleware to attach a request_id to every request (or use the X-Request-Id header if provided by the client/lb).
4. Replace console.* calls in backend with logger.* (info|warn|error).
5. Install @sentry/node + @sentry/react. Configure DSN via env. Instrument error handlers in backend (Express error middleware) and frontend (ErrorBoundary at the app root).
6. Install @opentelemetry/sdk-node and configure tracing. Export traces to the OTLP endpoint of Grafana Cloud (or local Jaeger in dev).
7. Define endpoints clearly:
 - GET /health - returns 200 if process is alive.
 - GET /ready - returns 200 only if Postgres, Redis, and Mongo (still present for legacy if any) all respond.
 - These are NOT auth-protected.
8. Add metrics:
 - Increment a counter for every API request labeled by method+path+status.
 - Histogram of request latency.
 - Counter of AI cost in cents per request.
 - Use prom-client (or OTel metrics).
 - Expose at /metrics (auth-protected with a static token).
9. Create docs/runbook.md with:
 - How to view logs (commands).
 - How to investigate a failed AI verification (check ai_jobs).
 - How to investigate a webhook failure (check audit_log + Stripe events).
 - On-call rotation note (placeholder).

```
# VERIFICATION
- Trigger an error; Sentry receives it within 30 seconds.
- Make 100 requests with k6 or similar; the /metrics endpoint shows the
  request count and latency histogram.
- Logs are JSON in prod mode and contain request_id, firm_id, user_id.
- /ready returns 503 if Postgres is killed; recovers when restored.

# COMMIT MESSAGE
feat(observability): pino logs, Sentry, OTel tracing, /metrics

- Structured JSON logs with request_id, firm_id, user_id propagation
- Sentry on backend (Express error mw) and frontend (ErrorBoundary)
- OpenTelemetry tracing exported to Grafana Cloud
- /health and /ready distinguished; /metrics for Prometheus scrape
- Runbook documenting common investigations
```

PROMPT 22 Deploy to AWS ca-central-1 (Terraform)

Provision the production infrastructure in AWS ca-central-1 via Terraform. Set up CI/CD to ECS Fargate.

What it does

The final prompt. Terraform provisions VPC, RDS Postgres Multi-AZ, ElastiCache Redis, S3 with Object Lock, ECR, ECS Fargate, ALB, CloudFront, WAF, Secrets Manager, and IAM roles. GitHub Actions builds and deploys on push to main. After this, you have a real URL to share with design partners.

PASTE THIS INTO CLAUDE CODE / CURSOR

```
# CONTEXT
We've finished all backend, frontend, and security work locally. Now we
provision AWS ca-central-1 infrastructure with Terraform and set up CI/CD
to deploy on every push to main.

# GOAL
- Terraform under infra/ provisions all AWS resources for staging + production.
- A Service Control Policy (SCP) restricts the AWS account to ca-central-1
  only - resource creation outside this region is denied.
- ECS Fargate runs the backend; a separate task runs the MCP server.
- ALB routes /api/* to backend, /mcp/* to MCP, otherwise to the static frontend
  served via CloudFront.
- RDS Postgres Multi-AZ, ElastiCache Redis, S3 buckets (documents, audit,
  retainer-pdfs all with Object Lock).
- AWS WAF with OWASP managed rule set.
- GitHub Actions pipeline: on push to main, build images, push to ECR,
  deploy to ECS staging, run smoke tests, then promote to production with
  manual approval.

# DO NOT TOUCH
- Do not point a real domain at this yet (use the ALB DNS name initially).
- Do not enable SSO or other deferred items.

# TASKS
```

1. Create infra/ structure:


```
infra/
  ■■■ modules/
  ■   ■■■ network/ (VPC, subnets, NAT, SG)
  ■   ■■■ data/    (RDS, ElastiCache, S3)
  ■   ■■■ compute/ (ECR, ECS, ALB, target groups)
  ■   ■■■ edge/    (CloudFront, WAF)
  ■   ■■■ secrets/  (Secrets Manager + KMS)
  ■■■ envs/
  ■   ■■■ staging/
  ■   ■■■ production/
  ■■■ README.md
```
 2. Each environment has its own backend.tf pointing to a unique S3 state bucket and DynamoDB lock table (manually create these once).
 3. Apply an Organization SCP that denies all actions outside ca-central-1 except for global services (IAM, CloudFront, Route53, etc.).
 4. Create the backend Dockerfile (multi-stage: build then prod) at apps/backend/Dockerfile. Likewise for MCP at apps/mcp-server/Dockerfile.
 5. Build static frontend output for production:


```
apps/consultant-web - vite build outputs to dist/.
```
 6. Create .github/workflows/deploy.yml:
 - On push to main:
 - a. Run all tests including cross-tenant.
 - b. Build Docker images for backend, MCP.
 - c. Build frontend dist.
 - d. Push images to ECR.
 - e. Sync frontend dist to S3 + CloudFront invalidation.
 - f. Update ECS service with new task definitions (rolling deploy).
 - g. Run smoke tests against staging.
 - h. Wait for manual approval, then promote to production.
 7. Set up secrets in AWS Secrets Manager:
 - JWT_ACCESS_SECRET, JWT_REFRESH_SECRET
 - STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET
 - ANTHROPIC_API_KEY, OPENAI_API_KEY
 - DATABASE_URL (constructed from RDS endpoint at deploy time)
 - REDIS_URL
 8. Backups:
 - RDS point-in-time recovery enabled, 35-day retention.
 - S3 versioning + Object Lock on documents and audit anchor buckets.
 9. Health/Ready checks:
 - ALB target group health check hits /ready.
 - ECS service grace period 60s.
- # VERIFICATION
- terraform plan in infra/envs/staging shows the expected resource set.
 - terraform apply succeeds.
 - Push a commit to main; pipeline deploys to staging.
 - Hit https://<alb-dns>/health and /ready - both 200.
 - Sign up a test firm via the deployed UI; the entire flow works:
 - create student, upload doc, AI verifies, generate retainer, pay invoice via Stripe test mode.
 - Manually approve promotion to production; everything moves over.

```
# COMMIT MESSAGE
feat(infra): Terraform AWS ca-central-1 + GitHub Actions deploy

- Modules: network, data, compute, edge, secrets
- Staging + production environments with separate state
- Org SCP locks region to ca-central-1
- ECS Fargate for backend + MCP; CloudFront + S3 for frontend
- GitHub Actions pipeline with manual prod promotion
- All secrets in AWS Secrets Manager + KMS
```

After Prompt 22

If you've executed all 22 prompts plus the 15.5 cleanup step and verified each one, you are now in a position most CRM founders take 12 months to reach. You have:

- A multi-tenant Postgres-backed application with RLS at the database level.
- Tenant-isolation tests that fail the build if cross-tenant data leaks.
- AI verification preserved verbatim from your existing work.
- MCP server connected to the new backend.
- Stripe Connect with trust ledger and e-signed retainers.
- shadcn/ui-based consultant and student portals with role-aware routing.
- Tamper-evident audit log + one-click CICC Inspection Packet.
- Full observability via pino + Sentry + OpenTelemetry.
- Production deployment in AWS ca-central-1 via Terraform + CI/CD.
- All data in Canada, all the time.

Next steps after Prompt 22

Reopen the Master Plan. You are now at the start of *Week 8 — UAT and launch*. Run a bug bash with your design partners on the deployed staging environment. Onboard them to production. Start collecting real refusal letters into the data flywheel. Begin the SOC 2 evidence collection (Vanta or Drata).

★ What's still ahead

Prompts 23+ (not in this document) cover the V2 features: PGWP/PR/Citizenship lifecycle UI, two-way WhatsApp inbound, white-label subdomains, SSO, mobile apps, IRCC GCKey integration. We write those prompts when the product hits the milestones described in the Master Plan: \$500K ARR for V2 features, \$1M+ for V3.

Ship Prompt 01 today. Don't read ahead. The plan only works if you execute it in order.